Introduction to Embedded Systems

Course Code: 244066

Chapter 7: STM32 Timer Interface

By

Dr. Muhammad Imran Nadeem

Zhengzhou University

Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

7.5. Timer Applications in Embedded Systems

7.6. Summary

7.1. Introduction to Timers in Embedded Systems

What is a Timer in Embedded Systems?

- A **timer** is a hardware module inside a microcontroller that **counts clock pulses** over time.
- It helps the CPU perform time-based tasks **without constant software checking**.
- A timer can **generate precise time delays**, **measure intervals**, or **trigger events** at fixed times.
- **Example:**
  - Turning an LED ON and OFF every 1 second.
  - Measuring how long a sensor signal stays HIGH.



7.1. Introduction to Timers in Embedded Systems

Why Are Timers Important?



Timers are essential for:

1. **Accuracy:** Perform tasks at exact time intervals.
2. **Automation:** CPU doesn't need to "wait"; timer can trigger actions automatically.
3. **Efficiency:** Frees the processor to do other work while timer keeps time.
4. **Synchronization:** Coordinate multiple tasks (like communication, sensor reading).

7.1. Introduction to Timers in Embedded Systems

Applications of Timers



- **Delays:**
  - Generate time delays without using CPU loops.
  - **Example**: LED blinks every 500 ms.
- **Event Scheduling:**
  - Trigger periodic actions, like reading a sensor every 100 ms.
- **PWM Generation:**
  - Create precise pulse signals for **motor control**, **LED brightness**, or **audio**.
- **Safety Monitoring:**
  - Watchdog timers **reset** the system if software freezes or fails to refresh them.

7.1. Introduction to Timers in Embedded Systems

Types of Timers in STM32

1. System Tick Timer (SysTick):

1. Built into the Cortex-M core.
2. Provides a simple, regular interrupt source (e.g., every 1 ms).
3. Commonly used for real-time operating systems (RTOS) tick count or delays.

2. Watchdog Timers:

1. Safety timers that reset the system if software stops responding.
2. Protects against system hang-ups or infinite loops.

3. Basic Timers (TIM):

1. Programmable hardware timers.
2. Used for delays, frequency measurement, PWM generation, etc.
3. Examples: TIM2, TIM3, TIM4, etc.

Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

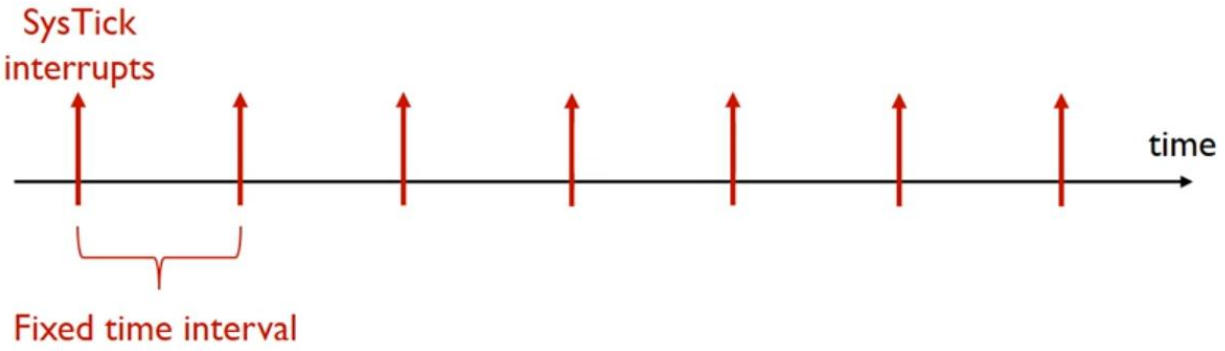
7.5. Timer Applications in Embedded Systems

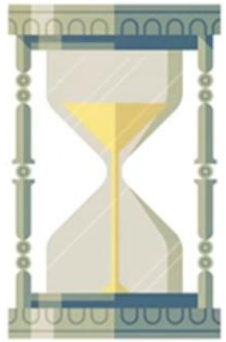
7.6. Summary

7.2. System Tick Timer (SysTick)

What is the SysTick Timer?

- **SysTick** = *System Tick Timer*
- It is a **built-in timer** inside every **ARM Cortex-M** processor.
- Provides a **simple, consistent time base** for delays, periodic tasks, and operating systems.
- Runs **independently from other peripherals,** no need to configure separate TIM modules.





7.2. System Tick Timer (SysTick)

Key Points:

- 24-bit **down counter** (counts from a value down to zero)
- Can generate **interrupts** at regular intervals
- Ideal for **timekeeping** or **scheduling tasks**
- **Example Uses:**
  - Creating 1 ms time base for delays
  - Generating “ticks” for an RTOS
  - Simple periodic LED blinking



7.2. System Tick Timer (SysTick)

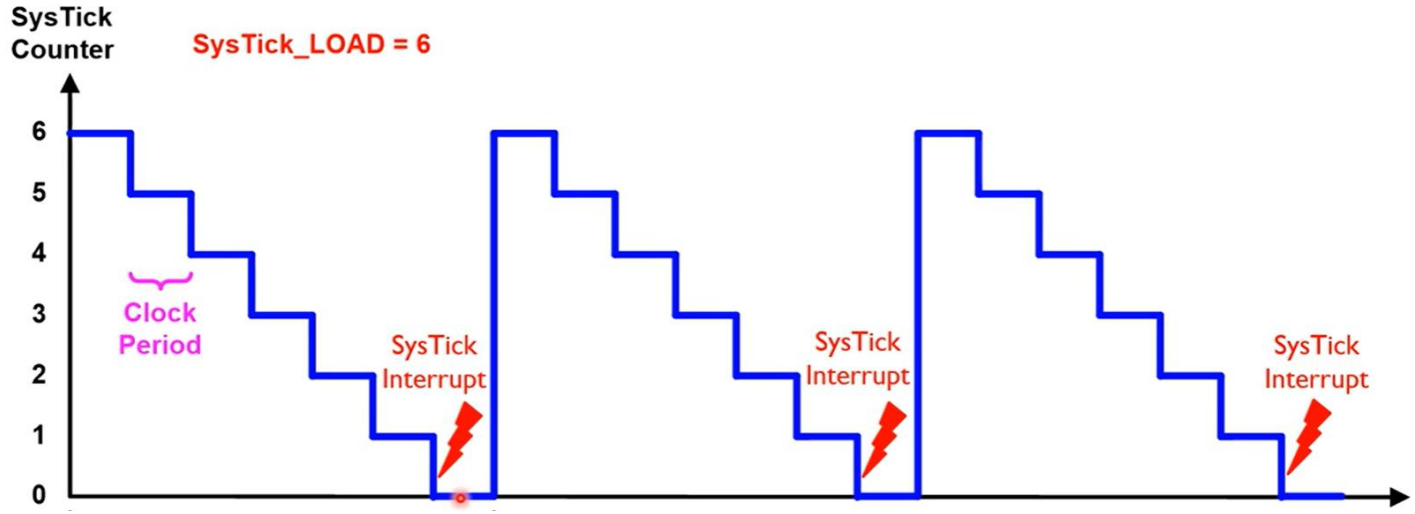
Purpose of the SysTick Timer

- Helps the CPU perform **repetitive tasks automatically**.
- **Saves** CPU effort as no need to manually count instruction cycles.
- Common uses:
  - Generate **regular time interrupts** (e.g., every 1 ms)
  - Create **software delays**
  - Maintain **system uptime counter**
  - Provide a **time base** for RTOS task switching

7.2. System Tick Timer (SysTick)

SysTick Timer Registers

- The SysTick timer has **three main registers**:



Register	Function	Description
CTRL (Control and Status)	Start/stop SysTick, check flags	Enables timer and interrupt
LOAD (Reload Value)	Sets the value to count down from	Determines the time period
VAL (Current Value)	Current counter value	Automatically decrements each tick

7.2. System Tick Timer (SysTick)

How SysTick Works (Operation Flow)

- Load initial value into LOAD register.
- Write any value to VAL register (clears counter).
- Enable SysTick in CTRL register.
- Counter starts **counting down** each clock tick.
- When it reaches **0**, it:
  - Sets the **COUNTFLAG** bit.
  - Generates an **interrupt** (if enabled).
  - Automatically **reloads** the value from LOAD and continues.
- **Cycle repeats automatically**

7.2. System Tick Timer (SysTick)

Example – Creating a Simple Delay
Blink an LED every 500 ms using SysTick.

Step 1: Configure SysTick

```
SysTick->LOAD = 72000000 / 1000 - 1; // 1 ms tick (assuming 72 MHz clock)
SysTick->VAL  = 0;                   // Clear current value
SysTick->CTRL = 7;                   // Enable SysTick, interrupt, use CPU clock
```

Step 2: Create Delay Function

```
void delay_ms(uint32_t ms) {
    for(uint32_t i = 0; i < ms; i++) {
        while(!(SysTick->CTRL & (1 << 16)));  // Wait for COUNTFLAG
    }
}
```

7.2. System Tick Timer (SysTick)

Step 3: Use in Main Program

```
while(1) {
    LED_ON();
    delay_ms(500);
    LED_OFF();
    delay_ms(500);
}
```

Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

7.5. Timer Applications in Embedded Systems

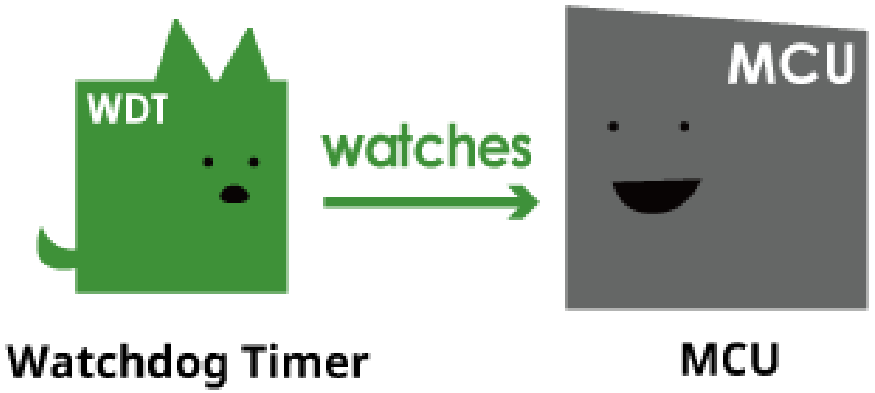
7.6. Summary

7.3. Watchdog Timer

What is a Watchdog Timer (WDT)?

- A **Watchdog Timer** is a **safety feature** built into embedded systems.
- It helps **detect and recover** from software errors or system crashes.
- If a program stops responding, the watchdog resets the system automatically.

The Watchdog ensures that your system never gets stuck forever, it always restarts to a known working state.



7.3. Watchdog Timer

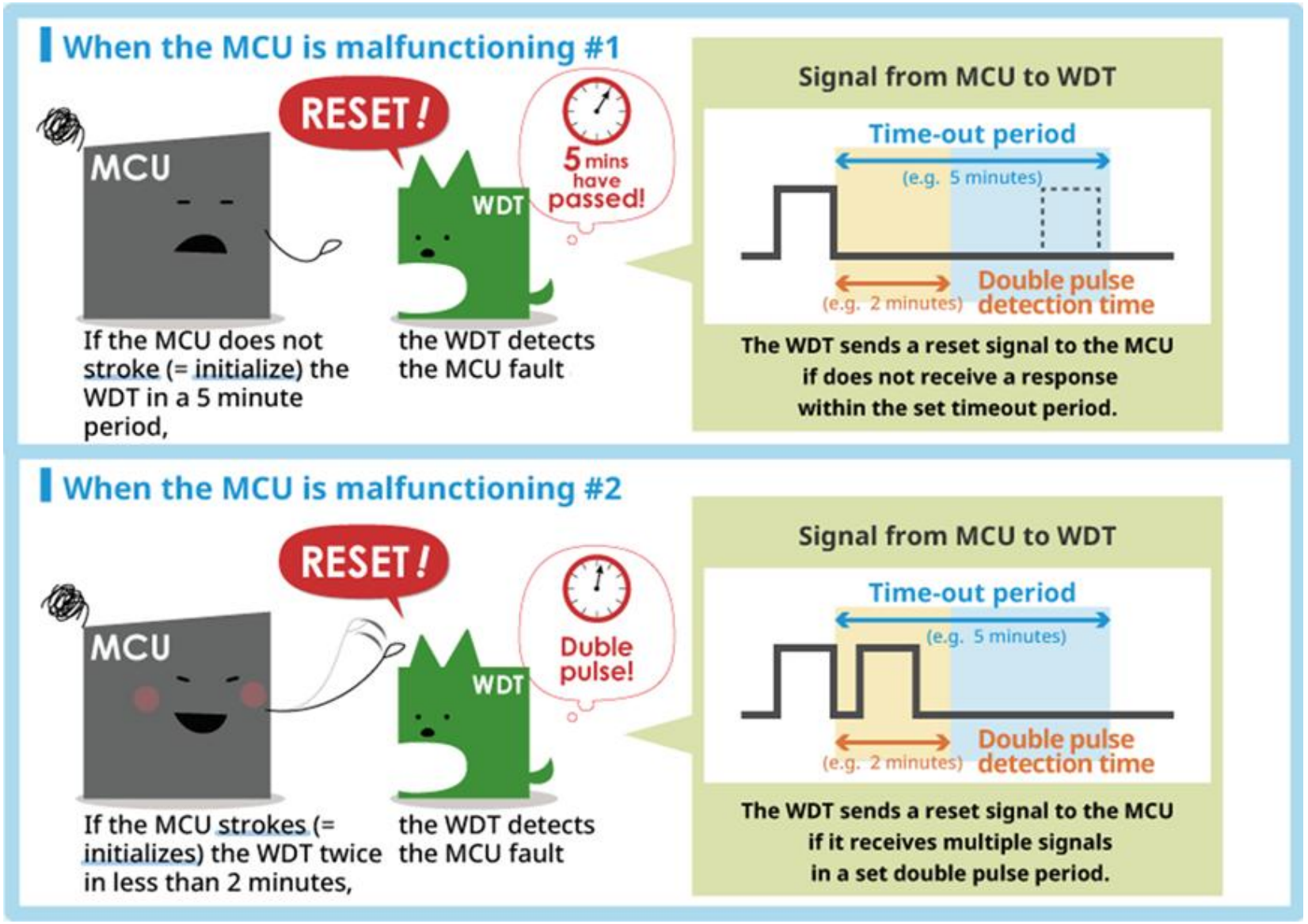
Purpose: System Reliability and Fault Detection

- The Watchdog timer ensures **continuous system operation** even when unexpected software problems occur.
- It detects:
  - **Infinite loops**
  - **Deadlocks** (program waiting forever)
  - **Software crashes**
- When these occur, it **resets the microcontroller** automatically.

Example:

- In an industrial controller, if a sensor reading routine hangs, the watchdog resets the MCU to restore normal operation.

7.3. Watchdog Timer



7.3. Watchdog Timer

Types of Watchdog Timers in STM32

STM32 microcontrollers have **two kinds** of watchdog timers:

- **Independent Watchdog (IWDG)**
  - Runs on its **own low-speed clock (LSI)**.
  - **Independent** from the main system clock → still works even if the main clock fails.
  - Cannot be stopped once started.
  - Used for **critical reliability** applications.
- **Window Watchdog (WWDG)**
  - Uses the **APB1 clock** (main system clock).
  - The program must refresh it **within a specific time window** (not too early, not too late).
  - More flexible, used for **precision timing supervision**.

7.3. Watchdog Timer

Key Registers and Configuration

Independent Watchdog (IWDG) Registers

Register	Function
IWDG_KR	Key Register – used to start and refresh watchdog
IWDG_PR	Prescaler Register – sets the clock divider
IWDG_RLR	Reload Register – sets the timeout value
IWDG_SR	Status Register – indicates update status

7.3. Watchdog Timer

How It Works:

1. Enable **LSI** clock.
2. Write key **0xCCCC** → starts watchdog.
3. Write key **0xAAAA** regularly → refresh (feed) the watchdog.
4. If not refreshed before timeout → MCU **resets**.

7.3. Watchdog Timer

Configuring the Independent Watchdog

- You must "feed" the watchdog before the counter reaches zero.
- If not, it assumes the program is stuck → automatic reset.

```
// 1. Enable LSI clock
RCC->CSR |= RCC_CSR_LSION;
while(!(RCC->CSR & RCC_CSR_LSIRDY));  // Wait for LSI ready

// 2. Configure Watchdog
IWDG->KR  = 0x5555;      // Enable access to PR and RLR
IWDG->PR  = 4;           // Set prescaler (divide clock)
IWDG->RLR = 1000;        // Set reload value (~1s timeout)

// 3. Start Watchdog
IWDG->KR  = 0xCCCC;      // Start the watchdog

// 4. In main loop, refresh regularly
while(1) {
    IWDG->KR = 0xAAAA;  // Refresh (feed the dog)
    do_some_task();
}
```

7.3. Watchdog Timer

Window Watchdog (WWDG) Configuration

- Uses main clock — faster than IWDG.
- You must refresh within a specific "window" of time.
  - Too early → error (reset).
  - Too late → timeout (reset).

Register	Function
WWDG_CR	Control Register (enable, counter value)
WWDG_CFR	Configuration Register (window value, prescaler)
WWDG_SR	Status Register (flags)

7.3. Watchdog Timer

Practical Example: Implementing Watchdog in STM32
Protect system from hangs while blinking an LED.

```
int main(void) {
    init_LED();
    init_IWDG();  // Configure IWDG as shown before

    while(1) {
        toggle_LED();
        delay_ms(500);
        IWDG->KR = 0xAAAA; // Refresh before timeout
    }
}
```

What happens if you comment out the refresh line?

- After timeout (~1 second), MCU **automatically resets**.
- LED blinking pattern restarts from the beginning → shows watchdog action.

Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

7.5. Timer Applications in Embedded Systems

7.6. Summary

7.4. Basic Timer (TIM)

What is a Basic Timer (TIM)?

- **Timer (TIM)** modules are **hardware counters** inside STM32 microcontrollers.
- They can **count clock pulses** and **generate time events automatically.**
- TIMs are called *"general-purpose timers"* because they can handle **many timing functions**:
  - Delay generation
  - Event counting
  - Pulse-width modulation (PWM)
  - Time measurement
- **Analogy:**

  A **digital stopwatch** built into your MCU

  it starts counting automatically when you start it.



7.4. Basic Timer (TIM)

General-Purpose Timers in STM32:

- STM32 has several timer modules:
  - **Basic Timers** – TIM6, TIM7 (simple counters, no input/output pins)
  - **General-Purpose Timers** – TIM2, TIM3, TIM4, etc. (used for most timing tasks)
  - **Advanced Timers** – TIM1, TIM8 (used for motor control, complex PWM)
- **Key Features:**
  - 16-bit or 32-bit counters
  - Can count **up**, **down**, or **up/down**
  - Can generate **interrupts** or **PWM signals**
  - Highly configurable for different applications

7.4. Basic Timer (TIM)

Example:

- TIM2 → delay generation
- TIM3 → PWM for LED brightness
- TIM4 → input capture (measure pulse width)



7.4. Basic Timer (TIM)

Main Registers of TIM

Each timer uses **several key registers** to control counting behavior and timing.

Register	Full Name	Function
PSC	Prescaler Register	Divides the input clock to adjust timing speed
ARR	Auto-Reload Register	Sets the maximum counter value (period)
CNT	Counter Register	Holds the current count value (changes as timer runs)
CCR	Capture/Compare Register	Stores compare value for interrupt or PWM output

7.4. Basic Timer (TIM)

How Timer Frequency is Calculated
Timer frequency depends on **system clock** and **prescaler settings**.

$$f_{timer} = \frac{f_{clock}}{(PSC + 1) \times (ARR + 1)}$$

- **Calculations:**
  - System Clock = 72 MHz
  - PSC = 71 → divides by 72
  - ARR = 999 → counts 1000 times
  - Timer overflow (event) every **1 ms**

7.4. Basic Timer (TIM)

Timer Counting Modes

- **Upcount Mode**
  - Counter starts from **0** → **ARR**
  - When it reaches ARR, it **overflows to 0** and sets an **update flag**.
  - Commonly used for periodic interrupts and delays.
- **Downcount Mode**
  - Counter starts from **ARR** → **0**
  - When it reaches 0, it **reloads ARR** and continues counting.
  - Used in some timing synchronization cases.
- **Up/Down Mode (Center-Aligned)**
  - Counter goes **0** → **ARR** → **0**, forming a triangle wave count.
  - Used in **PWM** for motor control (smooth output).

Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

7.5. Timer Applications in Embedded Systems

7.6. Summary

7.5. Timer Applications in Embedded Systems

Why Use Timers in Embedded Systems?

- Timers are like the **"timekeepers"** of a microcontroller.
- They help perform actions **accurately and automatically** based on time.
- Instead of using CPU "software delays," timers provide **hardware-based precision**.
- Timers can:
  - Create **delays**
  - **Measure signals**
  - **Trigger events periodically**
  - **Generate PWM** for control systems



7.5. Timer Applications in Embedded Systems

Application 1: Generating Precise Delays

- One of the **most common uses** of a timer is to create **accurate delays**.
- The timer counts clock cycles → when it reaches the set value, it signals "time's up."
- Unlike software loops, hardware timers are **not affected by instruction execution time**.
- **Steps:**
  - Set timer **prescaler (PSC)** and **auto-reload (ARR)**.
  - Start the timer.
  - Wait for **update flag** or **interrupt** when time is complete.
- **Example:**
  - Delay of 1 second using TIM2 with 72 MHz system clock.

```
TIM2->PSC = 7200 - 1;     // 10 kHz timer clock
TIM2->ARR = 10000 - 1;    // 1s overflow
while(!(TIM2->SR & 1));   // Wait for overflow flag
TIM2->SR &= ~1;           // Clear flag
```

7.5. Timer Applications in Embedded Systems

Application 2: Measuring Input Signals (Pulse Width)

- Timers can measure the **duration of an input pulse** (HIGH or LOW signal).
- This is done using **Input Capture Mode**.
- The timer records the counter value at the moment a signal **edge** (rising/falling) occurs.
- The difference between two captured values = **pulse width or period**.
- **Use Cases:**
  - Ultrasonic distance sensors (measure echo pulse width).
  - Speed sensors (measure pulse frequency).
  - IR signal decoding.

7.5. Timer Applications in Embedded Systems

Application 3: Periodic Event Triggering

- Timers can **generate periodic interrupts** to trigger tasks automatically.
- Commonly used for:
  - Reading sensors at fixed intervals
  - Updating display every few milliseconds
  - Sampling data in real-time systems
- **Concept:**
  - Timer generates an **interrupt** each time it overflows.
  - The CPU executes an **Interrupt Service Routine (ISR)** each time.t

7.5. Timer Applications in Embedded Systems

Application 4: PWM Signal Generation

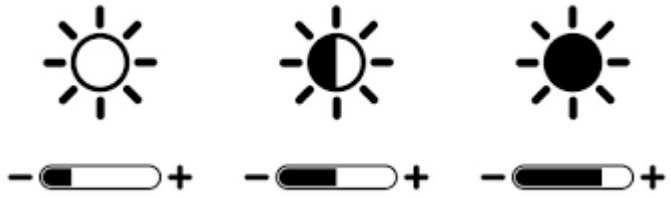
- **PWM (Pulse Width Modulation)** = output signal where **duty cycle** represents control value.
- Timer uses **Compare Register (CCR)** to define ON time within one period (ARR).
- Output pin toggles HIGH when CNT < CCR.
- **Example**:
  - LED Dimming: Adjust brightness by changing duty cycle.
  - Motor Control: Control speed or torque.
  - Servo Control: Adjust angle via pulse width.

7.5. Timer Applications in Embedded Systems

PWM in Real-Life Applications

- PWM signals are widely used in **control systems**:
  - **LED brightness** → Adjusts perceived light intensity.
  - **DC Motor speed** → More ON time = faster motor.
  - **Servo motor position** → Pulse width determines angle.
  - **Sound generation** → PWM audio signal.
- **Analogy:**

  PWM is like **switching a light ON and OFF very fast** — the longer it stays ON, the brighter it appears.





Contents

7.1. Introduction to Timers in Embedded Systems

7.2. System Tick Timer (SysTick)

7.3. Watchdog Timer

7.4. Basic Timer (TIM)

7.5. Timer Applications in Embedded Systems

7.6. Summary

7.6. Summary

Introduction to Timers

- Timers are **hardware counters** inside the STM32 used for time-based operations.
- They can **count clock pulses**, **measure intervals**, **generate delays**, and **trigger periodic events**.
- Essential for **real-time control**, **signal measurement**, and **system synchronization**.

System Tick Timer (SysTick)

- A **built-in 24-bit timer** in the ARM Cortex-M core.
- Provides a **regular time base** (e.g., 1 ms tick) for delays and scheduling.
- **Operation:** counts down → reaches zero → sets flag → optional interrupt.
- Commonly used for **software delays** or **RTOS task timing**.

7.6. Summary

Watchdog Timer (WDT)

- Ensures **system reliability and fault recovery**.
- Automatically **resets MCU** if program hangs or fails to refresh it.
- **Types:**
  - **IWDG (Independent Watchdog):** runs on LSI clock, always active once started.
  - **WWDG (Window Watchdog):** uses main clock, refresh must occur within a set window.
- **Registers:** Key (KR), Prescaler (PR), Reload (RLR), Status (SR).
- Commonly used in **safety-critical systems**.

7.6. Summary

Basic Timer (TIM)

- **General-purpose timers (TIM2, TIM3, TIM4, etc.)** can generate time delays, measure signals, or produce PWM.
- **Key Registers:**
  - PSC – Prescaler (divides input clock)
  - ARR – Auto Reload (sets counting limit)
  - CNT – Counter (current value)
  - CCR – Capture/Compare (defines compare or PWM output)
- **Counting Modes:** Upcount, Downcount, Up/Down (center-aligned).
- **PWM Generation:** Output stays HIGH while CNT < CCR.

7.6. Summary

Timer Applications

- **Precise Delays:** Replace software loops for accurate timing.
- **Input Signal Measurement:** Use input capture to measure pulse width or frequency.
- **Periodic Triggering:** Generate interrupts at fixed intervals for repetitive tasks.
- **PWM Output:** Control motor speed, LED brightness, or servo position.

Practical Implementation Examples

- **SysTick Delay Function** – LED blinking at 1-second intervals.
- **Watchdog Example** – Auto-reset when loop fails to refresh timer.
- **PWM Example** – LED brightness or motor speed controlled via duty cycle.
- **Periodic Interrupt Example** – Reading a sensor every 10 ms.

